File Processing Platform — Detailed Project Specification

Product requirements • functional scope • engineering challenges • implementation direction

1. What Are We Actually Building?

A production-style platform where users upload files, select useful processing operations, and receive processed results. The product is not primarily about uploading files; the core is a reliable processing pipeline that can handle work that may be slow, fail, or arrive in large quantities.

```
Upload file → choose operation → create processing job → process file → store result →
notify user → download result
```

The project starts simple and becomes more sophisticated as requirements force us to introduce queues, workers, object storage, retries, caching, rate limiting, and other engineering concepts.

2. Why Would Someone Use It?

The platform saves users from installing or running separate tools for common file operations. A user can upload a file, select an operation, submit it, leave the page, and later return to retrieve the result.

Example: a user uploads a 300-page PDF and selects “Extract text”. The API accepts the job quickly, processing happens in the background, and the user is notified when the resulting text file is ready.

3. User Accounts

Users can register, log in, log out, view/update their profile, and access only their own files and jobs. Later we can introduce user/admin roles.

Authentication and authorization are deliberately part of the project because the application contains private files.

4. File Upload

Initial supported types: PDF, JPG/PNG/WebP images, and CSV. The upload flow records filename, MIME type, size, owner, upload timestamp, and a storage key. The server should validate file type and size before accepting it.

We should distinguish the logical file record from the physical file. The database knows what the file is and who owns it; storage holds the actual bytes.

5. Where Do The Files Live?

During local development, the simplest approach is a dedicated uploads directory on the server. This lets us learn the upload/processing flow without introducing cloud infrastructure immediately.

For production, actual files should live in object storage such as AWS S3. PostgreSQL should store metadata and a storage key rather than the binary contents.

```
PostgreSQL: file_id = 123 name = report.pdf size = 42 MB storage_key = uploads/123/report.pdf
Object storage: actual report.pdf bytes
```

This separation matters because databases are not the ideal place to store large arbitrary files, while object storage is designed for that job.

6. File Management

Users can list files, search by name, filter by type, sort by name/date/size, view details, rename files, download files, and delete files.

Deleting a file should eventually involve both the database record and the physical object. This creates an opportunity to learn about consistency and failure handling.

7. What Does “Processing” Mean?

Processing means the application actually reads or transforms the uploaded file using a suitable library or program.

PDF example: report.pdf → parse PDF → extract text → produce report.txt.

Image example: photo.png → resize to 800×600 → produce resized image.

CSV example: data.csv → validate rows → produce a cleaned/validated CSV and a validation report.

Parsing is simply one type of processing: reading structured file contents so the application can work with the information inside.

8. Initial Processing Operations

PDF: extract text, read metadata, merge PDFs, split PDFs, compress PDFs, generate previews/thumbnails.

Images: resize, convert formats, compress, generate thumbnails.

CSV: validate structure, detect malformed rows, generate summary information, convert formats.

We do NOT need to implement every operation. The MVP should start with one or two operations and expand only when useful.

9. Processing Jobs

Every processing request becomes a job. A job records the input file, requested operation, status, timestamps, progress if available, output file, and error information.

```
QUEUED → PROCESSING → COMPLETED ■ FAILED ■ CANCELLED
```

The user should be able to see job history rather than wondering whether an operation is still running.

10. Why A Message Queue?

The queue is not the product. It solves a real problem: some file operations can be slow and many users may request them at the same time.

Instead of holding an HTTP request open while a 500-page PDF is processed, the API creates a job and returns. A worker later takes the job from a queue and processes it.

```
User → API → create job → Queue → Worker → File processor → Result
```

If 100 jobs arrive, they can wait in the queue while available workers process them. This also gives us a natural place to implement retries and controlled processing.

11. Workers

A worker is a background process responsible for taking a job and doing the actual file-processing work. We can start with one worker and later run multiple workers if required.

This is where we will learn concepts such as background jobs, retries, concurrency, and idempotency rather than studying them in isolation beforehand.

12. Job Status & Progress

The frontend should display states such as queued, processing, completed, and failed. Actual percentage progress can be added only for operations where reliable progress can be calculated.

The API can expose an endpoint such as GET /jobs/:id so the frontend can retrieve the current state.

13. Results

A successful processing operation produces a result file or another useful output. The result should be stored and associated with the processing job.

```
Original file → Job → Result file
```

For example, report.pdf may produce report.txt after text extraction. Users should be able to download the result and see which operation produced it.

14. Failure Handling

Processing can fail because of corrupt files, unsupported formats, malformed CSV data, worker crashes, insufficient resources, or programming errors.

The system should record the failure rather than silently losing the job. Later we can introduce controlled automatic retries, maximum retry counts, and a failed-job state.

We will also learn idempotency: if a job is accidentally delivered to a worker twice, the system should avoid producing incorrect duplicate side effects.

15. Notifications

When a job finishes or fails, the user should receive an in-app notification. Email can be added later.

```
Job completed → create notification → user sees “Your report.pdf processing is complete.”
```

16. File Sharing

Optional feature: allow a user to generate a temporary share link for a file. The link can expire or be revoked. This creates useful security and authorization considerations.

17. Storage Quotas & Limits

Users can have limits such as maximum storage, maximum file size, allowed file types, and maximum processing requests. These are real business rules rather than arbitrary technical features.

```
Storage used: 2.4 GB / 5 GB Maximum file size: 100 MB Allowed: PDF, PNG, JPG, CSV
```

18. Search & Filtering

Users should be able to search files and jobs and filter by file type, job status, date range, and other metadata. This gives us meaningful SQL querying, indexes, pagination, and filtering requirements.

19. Admin Panel

Admins can view users, files, processing jobs, failed jobs, storage usage, and basic processing statistics. They can disable users, remove problematic files, and retry failed jobs.

Admin functionality should be added after the core user workflow works.

20. Audit Log

Important actions should be recorded, such as upload, delete, rename, job creation, job completion/failure, sharing, download, and admin actions.

```
08:42 — user uploaded report.pdf 08:43 — processing job created 08:44 — job completed 08:44 — result downloaded
```

21. Dashboard

A simple dashboard can show total files, storage used, processing jobs, completed/failed counts, and recent jobs. It does not need to be a sophisticated analytics product.

22. Target Architecture

This is the eventual direction, not the architecture you should blindly copy. You will first create your own high-level design and we will review it.

```
Frontend ↓ REST API ■■■ PostgreSQL → users / files / jobs / metadata ■■■ Object Storage →
actual files ■■■ Queue → Workers → File Processing → Result Storage
```

23. Technology Direction

Frontend: React/Next.js + TypeScript. Backend: Node.js + TypeScript + Express/NestJS. Database: PostgreSQL. Local file storage initially. Production object storage such as AWS S3. Queue/worker technology and Redis will be selected when the actual requirements justify them. Docker and cloud deployment come later.

24. MVP — What We Build First

The first version should be intentionally small: registration/login, upload/list/download/delete, one or two processing operations, processing-job records, job status, result storage, and a basic UI.

We should NOT start with Redis, multiple workers, email, admin dashboards, sharing, quotas, or ten processing formats. Those are later milestones.

25. Development Progression

Stage 1: basic full-stack file upload and metadata.

Stage 2: implement one real processing operation synchronously.

Stage 3: move slow processing into background jobs.

Stage 4: introduce a queue and worker.

Stage 5: move production files to object storage.

Stage 6: add retries, failure handling, idempotency, limits, and security.

Stage 7: add Redis/caching/rate limiting only where justified.

Stage 8: Dockerize and deploy.

Stage 9: test, optimize, document, and prepare the project for interviews.

26. What This Project Should Teach You

Backend API design, PostgreSQL/database design, file uploads, object storage, parsing/file libraries, asynchronous processing, queues, workers, retries, concurrency/idempotency, authentication, authorization, rate limiting, caching, testing, Docker, deployment, logging, and practical system-design decisions.

27. Resume Goal

The finished project should be something you can explain deeply. An interviewer should be able to ask why files are not stored in PostgreSQL, why processing is asynchronous, what happens when a worker fails, how duplicate processing is prevented, how the system scales, and where Redis helps—and you should be able to answer from your own implementation.

28. What We Are NOT Building

Not Dropbox, Google Drive, Adobe Acrobat, an enterprise document-management suite, or an AI-first application. AI is optional and can be added later only if it provides a genuinely useful processing capability.

29. Immediate Next Step

Do not code yet. Create your own high-level system design from the functional requirements. Use boxes and arrows; it does not need to be pretty. Show the components you think we need, what data is stored where, and how a file moves through the system. Then we will review and refine it before implementation.